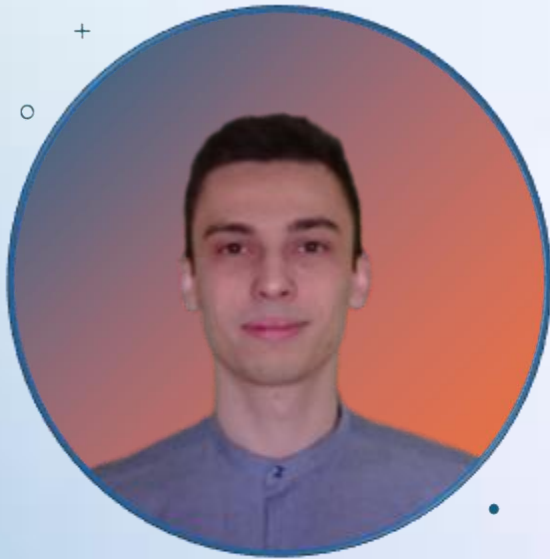




Advanced Debugging in IntelliJ IDEA

About me



Software Engineer at Intellias

Co-organizer of Codeus Community

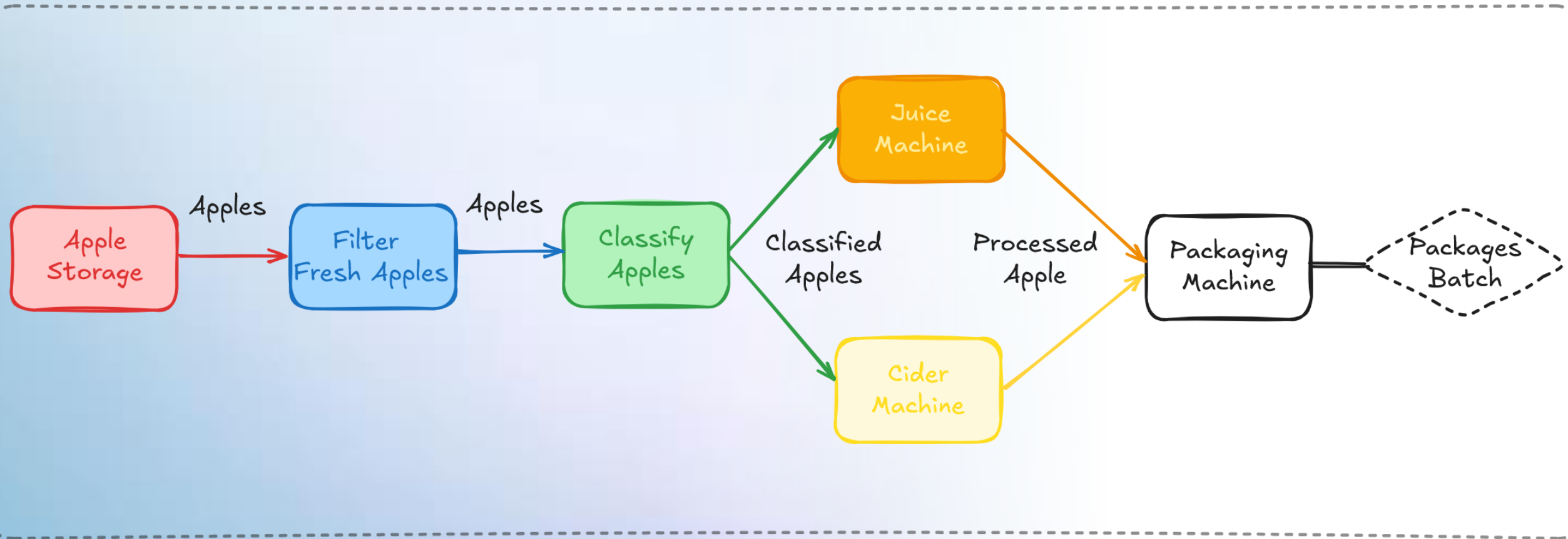
I like complex things like distributed systems
but I don't work on them...

... perhaps the main reason why I like
them.

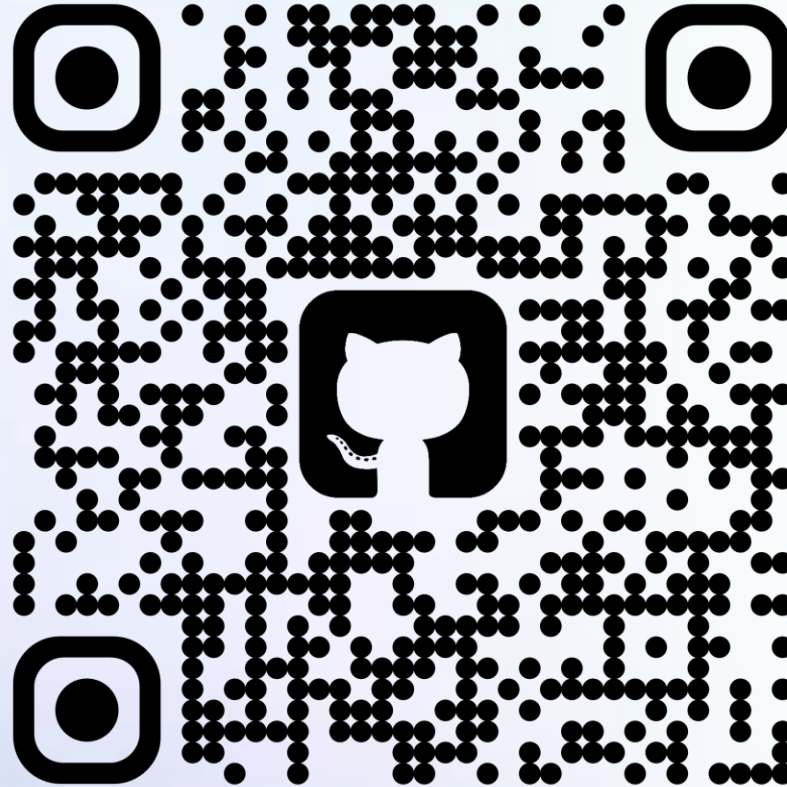
Target project

Target project

Apple Products Factory



Project GitHub QR-code



Minimal Debugging Set

Minimal Debugging Set

- Debug mode
- Breakpoints and Breakpoint Dialog
- Frame stack
- Variables view
- Evaluate expression

Minimal Debugging Set Summary

- pay attention on **labels** and **buttons** in IntelliJ IDEA UI
- the Evaluate Expression Dialog has a **history of previous expressions**
- use Inline Expression Evaluation in **code editor**
- in Variables View you can change **values of variables**
- in the Frame Stack you can **traverse** and **drop frames**
- remember, frame drop **doesn't reset** the app **state**
- use Breakpoint List Dialog to **organize** your **breakpoints**

More advanced stuff

Breakpoints

Breakpoints

- all Breakpoints live in **one workspace**
- a Breakpoint Marker can be used to **log messages** and be a **reference point** for other Breakpoints
- **restore removed** Breakpoint in the Breakpoint Dialog or IDE header menu
- use Dependent Breakpoints to build **granular** and **performant debug flow**

Navigation and Changing Execution Behavior

Navigation and Changing Execution Behavior

- use Run to Cursor to **quickly proceed to the required line** and reduce number of Breakpoints
- use Step Out to **leave method** but **let the method execution happen**
- use Force Return to **immediately leave method** with a **custom return value**
- use Throw Exception to **throw any exception** from the current frame
- use Reload Changed Classes (HotSpot) to **quickly update app sources with new logic**
- don't forget **to leave the Frame** for which you making changes before calling Reloading Changed Classes

Object Tracking

Object Tracking

- use Show Referring Objects to check what **objects holds reference to target one**
- use Labels to **track target object** within **different contexts**
- a Labeled Object can be used in **Breakpoint conditions**
- remember Label **lives within one debug session**

Extra

Extra

1. Watches
 - Basic watcher
 - Class level watcher
2. Renderers
3. Other Breakpoint types
 - Field breakpoint
 - Method breakpoint
4. Breakpoint Intentions
5. Async Stack
6. ... and more

Say "I don't have time to practice"

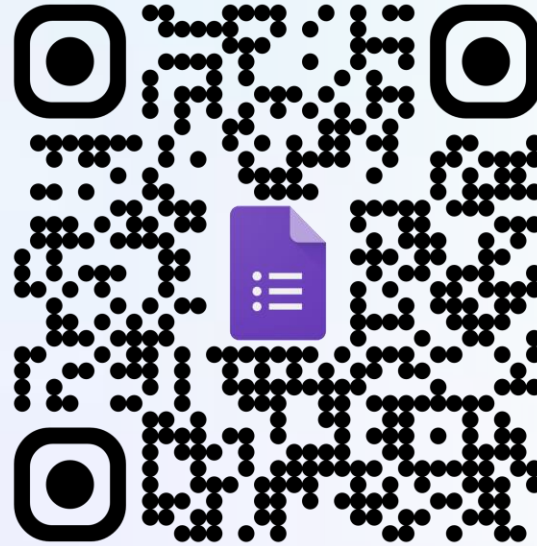


Say it. One. More. Time. MF

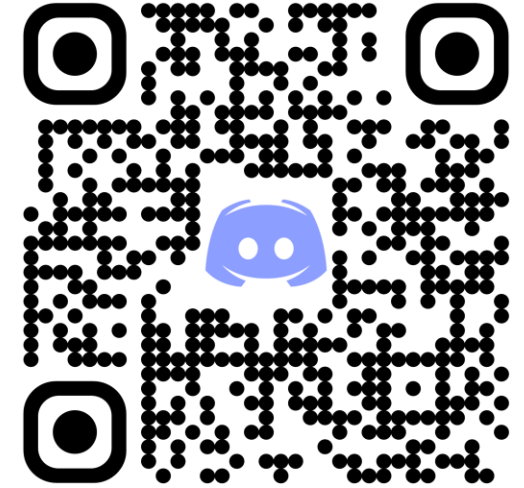
Thank you!



[YouTube Playlist](#)



[Share your feedback!](#)



[Codeus Community](#)



[Target Project](#)